Implementation Details for CaΣoS: A nonlinear sum-of-squares optimization suite

Torbjørn Cunis and Jan Olucak

September 2024

1 Introduction **1**

2 Important Information **2**
2.1 Overview of Toolboxes and Versions 2
2.2 Reproduction of Results 2
2.3 Result Inspection 3

3 Benchmark Details **3**
3.1 Benchmark I: Region-of-Attraction Estimation 3
3.2 Benchmark II: Backwards Reachable Set Estimation 5
3.3 Benchmark III: N-link pendulum ROA 6

4 Information About Other Toolboxes **8**
4.1 Toolbox Modifications 8

5 Benchmark Execution **9**

1 Introduction

This document is supplementary material for the paper CaΣoS: A nonlinear sum-of-squares optimization suite [1][1]. In Section 2 we provide information on how to reproduce the results provided in the paper. Section 3 provides more detailed explanations and algorithms for the three benchmarks. This also includes the used polynomial degrees. In Section 4 we provide tables with the modifications in the other toolboxes, and in Section 5 we provide simplified Matlab code listings to describe the measurement procedure.

[1]Submitted to the annual American Control Conference 2025.

2 Important Information

This section provides some important information in case the benchmark results shall be re-computed, or one only wants to inspect them. We first provide the toolboxes and versions used for the benchmark. Second, we provide information on reproducing the results, followed by instructions on how to use the saved benchmark results.

2.1 Overview of Toolboxes and Versions

Name	**Source**	**Version**
CaΣoS	*https://github.com/iFR-ACSO/casos*	v1.0
YALMIP	*https://yalmip.github.io/download/*	R20230622
SOSTOOLS 4.0	*https://github.com/oxfordcontrol/SOSTOOLS*	v4.02
SOSOPT	*https://github.com/SOSAnalysis/sosopt*	v2.0
SPOTless	*https://github.com/spot-toolbox/spotless*	-[2]

Table 1: Overview of used toolboxes

2.2 Reproduction of Results

If the interested reader wants to re-run the benchmarks (provided in *casos-benchmark_ACC25.zip*), run only certain parts of them, or inspect the implementation, please follow the instructions/information below.

- For CaΣoS, please follow the installation instructions on *https://github.com/iFR-ACSO/casos*. CaΣoS requires CasADi version 3.6.0, which must be on the Matlab path.
- We used Mosek version 10.1.21 for the benchmarks.
- Make sure, you only use the version of the other toolboxes provided (in the *otherFrameworks* folder). We made some small modifications as already mentioned in the paper. See Section 4 for details about the changes. If you already have different versions on your Matlab path, remove those in advance to avoid confusion or errors. If you run a benchmark or parts of it, they are added to the path of the current Matlab session.
- We ran each benchmark five times. This number can be easily changed to, e.g. one to check the functionality.
- Please see the comments in the source code as they might provide additional information regarding the implementation.

[2] No releases published.

Running a Benchmark

1. Add the CaΣoS root folder to your Matlab path (if not already done).
2. Go to folder *benchmark*
   - The subfolders for each toolbox contain the corresponding implementation of the benchmark problems.
   - Three *.mat* files are provided that conatin the final results of the corresponding benchmark.
   - For each benchmark a script exists.
3. Execute the desired benchmark script.

If you encounter problems or have questions or comments regarding the results and/or implementation please contact the corresponding authors! Please keep also in mind that computation times on different machines differ and that some of the benchmarks might take rather long times!

2.3 Result Inspection

If the reader is only interested in inspecting the results, we provide *.mat* files with the benchmark (final) results. For example, we provide the arrays that provide the computation times for each single run. For plotting, load the corresponding *.mat* file to the Matlab workspace. In the corresponding benchmark script, execute the plotting section or use data for custom manipulation or further inspection.

3 Benchmark Details

In this section, we provide more details about the benchmark examples. This includes a summary of the algorithm, the polynomial degrees and further information.

3.1 Benchmark I: Region-of-Attraction Estimation

The first benchmark deals with the computation of an inner-estimate of the ROA for the longitudinal motion of an aircraft with four states. In [2] both the nonlinear polynomial dynamics and the nonconvex SOS program are derived in terms of a three-step V-s-iteration algorithm (coordinate descent). Thereby, a shape function is used to increase the size of the ROA estimate of the iterations. The V-s-iteration algorithm reads [2]

1. γ-step: Solve for γ^* and s_1^* with V fixed as

$$(\gamma^*, s_1^*) \in \underset{\gamma\in\mathbb{R}, s_1\in\Sigma[x]}{\arg\max}\ \gamma$$
$$\text{s.t. } s_1(V^* - \gamma) - \nabla_x V^* f - l(x) \in \Sigma[x]$$

2. β-step: Solve for β^* and s_2^* with V, γ^* fixed as

$$(\beta^*, s_2^*) \in \underset{\beta\in\mathbb{R}, s_2\in\Sigma[x]}{\arg\max}\ \beta \qquad \text{s.t. } s_2(p-\beta)+\gamma^*-V^* \in \Sigma[x]$$

3. V-step: Solve for V with $s_1^*, s_2^*.\gamma^*, \beta^*$ fixed s.t.

$$\begin{aligned} V - l(x) &\in \Sigma[x] \\ s_1^*(V-\gamma^*) - \nabla_x V f - l(x) &\in \Sigma[x] \\ s_2^*(p-\beta^*)+\gamma^* - V &\in \Sigma[x] \end{aligned}$$

4. Repeat steps 1–3 as long as β continues to increase.

where $f \in \mathbb{R}[x]^4$ are the polynomial aircraft dynamics from [2], $p = 100\|x\|_2^2$ is the used shape function, $\beta > 0$ describes the size of the shape function, $\gamma > 0$ is the sublevel set of the Lyapunov function (candidate) $V \in \mathbb{R}[x]$, the linear operator $\nabla_x : \mathbb{R}[x] \to \mathbb{R}[x]^n$ is the polynomial derivative (with respect to the state x), $l(x) = 10^{-6}\|x\|_2^2$ ensures positivity, and $s_1, s_2 \in \Sigma[x]$ are SOS multipliers.

The above algorithm terminates if either $|\beta^{*,k} - \beta^{*,k-1}| \leq \epsilon$ (here $\epsilon = 10^{-3}$), a maximum number of iterations (here 100) is reached, or one of the steps above is infeasible. For all toolboxes the initial interval for the bisection is set to be $[0\ \ 1000]$.[3] All toolboxes are executed in the default options except SOSTOOLS where the simplification based on Newton polytopes is also turned on. We rely on MOSEK as the underlying SDP solver. The degrees of the polynomial decision variables V, s_1, and s_2 are given in Table 2.

Remark 1 *CaΣoS,* SOSTOOLS*, and* SOSOPT *allow to define SOS decision variables in the Gram form, i.e.,* $\Sigma[x]$*, whereas for* YALMIP *and* SPOTLESS *all polynomial decision variables are regular polynomials, i.e.* $\mathbb{R}[x]$*. Hence, depending on how the different toolboxes define it the degrees must be adapted accordingly.*

Table 2: Benchmark I polynomials and their degrees.

Polynomial	Type	min deg.	max deg
V	$\mathbb{R}[x]$	2	4
s_1	$\Sigma[x]$ or $\mathbb{R}[x]$	1 or 2	2 or 4
s_2	$\Sigma[x]$ or $\mathbb{R}[x]$	0	2 or 4

[3] Depending on a specific toolbox bisection works with a modified interval $[-1000\ \ 0]$. Signs in constraint might be adapted accordingly.

3.2 Benchmark II: Backwards Reachable Set Estimation

The second benchmark problem deals with computing the backward reachable of the longitudinal motion of an aircraft as presented in [3, Section V.B] for a time horizon $T = 3s$ without disturbances. The problem uses an adapted version of polynomial aircraft model found in [2].

The primary goal is to synthesize a backward reachable set in continuous time that reads $\Omega^V_{t,\gamma} = \{x \in \mathbb{R}^n \mid V(x,t) \leq \gamma\}$ where $V : \mathbb{R}^n \times \mathbb{R} \to \mathbb{R}$ is a so-called *reachability storage function*. Additionally, the goal is to synthesis a control law (with respect to control constraints) $u(t) = K(t,x)$ that ensures the system's trajectories remain within a pre-defined target tube $\Omega^r_{t,0} := \{x \in \mathbb{R}^n \mid r(t,x) \leq 0\}$.

The goal is to maximize the volume of the backward reachable set at $t_0 = 0s$ which is approximately solved using a bisection and a growth constraint in the V-step below. To ensure an inner-approximation of the backward reachable set, several set-containment conditions are derived in [3, Section V.B]. $\Omega^V_{t,\gamma} \subseteq \Omega^r_{t,0}$ ensure state constraint satisfaction for all $t \in [0,T]$, whereas $\Omega^V_{t,\gamma} \subseteq \{x \in \mathbb{R}^n \mid A(t,x)K(t,x) \leq b(t,x)\}$ for all $t \in [0,T]$ ensures control constraint satisfaction where $A(t,x) \in \mathbb{R}^{n\times m}[t,x]$ and $b(t,x) \in \mathbb{R}^n[t,x]$ are matrices of a time-varing polytop that describes the control constraints. To ensure reachability $\Omega^V_{t,\gamma} \subseteq \{x \in \mathbb{R}^n \mid \dot{V}(t,x) \leq 0\}$ for all $t \in [0,T]$. Such set-containment conditions can be rewritten into a bilinear SOS problem by applying the generalized $\mathcal{S}$-procedure [4].

The two-step coordinate descent algorithm is outlined below, which corresponds to optimization problem I and/or Algorithm I in [3, Section III.C].

1. γ-step: Solve for $\gamma, s_1, s_2, s_3, s_4, s_{51}, s_{52}, s_{61}, s_{62}, K$ and fix V^*

$$
\begin{aligned}
&s_1, s_2, (s_3 - \epsilon), s_4, s_{51}, s_{52}, s_{61}, s_{62} \in \Sigma[x,t] \\
&s_1(V^* - \gamma) - s_2 h - \nabla_t V^* - \nabla_x V^* f(x,K) \in \Sigma[x,t] \\
&- s_3 r_T + V^* - \gamma - s_4 h \in \Sigma[x,t] \\
&K - u_{\min} - s_{51}(V^* - \gamma) - s_{61} h \in \Sigma[x,t] \\
&u_{\max} - K - s_{52}(V^* - \gamma) - s_{62} h \in \Sigma[x,t]
\end{aligned}
$$

2. V-step: Solve for V and fix $\gamma^*, s_1^*, s_3^*, s_{51}^*, s_{52}^*$ and K^*

$$
\begin{aligned}
&s_0 \in \Sigma[x] \\
&s_1^*(V - \gamma^*) - s_2 h - \nabla_t V - \nabla_x V f(x,K^*) \in \Sigma[x,t] \\
&- s_3^* r_T + V - \gamma - s_4 h \in \Sigma[x,t] \\
&K^* - u_{\min} - s_{51}^*(V - \gamma^*) - s_{61} h \in \Sigma[x,t] \\
&u_{\max} - K^* - s_{52}^*(V - \gamma^*) - s_{62} h \in \Sigma[x,t] \\
&s_0(V^* - \gamma^*) + \gamma^* - V
\end{aligned}
$$

3. Repeat step 1 and 2 for 10 iterations[4].

where $h = (t - t_0)(T - t) \geq 0$ ensures continuous-time constraint satisfaction, $u_{\min}$ and $u_{\max}$ are the minimum and maximum control input respectively, and in this example $r_T \in \mathbb{R}[x]$ describes the terminal set. For the detailed parameters, the interested reader is referred to [3, Section V.B].[5] The used polynomial degrees are provided in Table 3.

Table 3: Benchmark II polynomials and their degrees.

Polynomial	Type	min deg.	max deg
V	$\mathbb{R}[t,x]$	0	4
K	$\mathbb{R}[t,x]$	0	4
s_0	$\Sigma[x]$ or $\mathbb{R}[x]$	0	2 or 4
s_1	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_2	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_3	$\Sigma[x]$ or $\mathbb{R}[x]$	0	2 or 4
s_4	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_{51}	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_{52}	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_{61}	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4
s_{62}	$\Sigma[t,x]$ or $\mathbb{R}[t,x]$	0	2 or 4

3.3 Benchmark III: N-link pendulum ROA

The third benchmark is used to test the scaling properties with respect to the number of states. Here, we make use of an N-link pendulum, where each problem has $n = 2N$ states. The dynamics are pre-computed up to $N = 4$. We use a Taylor series expansion to obtain the correpsonding polynomial dynamics.

For each problem, we pre-computed LQR controller where $Q \in \mathbb{R}^{n\times n}$ and $R \in \mathbb{R}^{m\times m}$ are both identity matrices for all problems. We used the solution of the Riccati equation $P \in \mathbb{R}^{n\times n}$ to compute the initial Lyapunov candidate. Depending on N, we load the pre-computed closed-loop dynamics and the initial Lyapunov function.

Compared to the first benchmark, we simplify the problem, by fixing $\gamma_1 = 1$, i.e., we only perform one bisection. The adapted V-s-iteration algorithm reads

[4] We just used 10 to limit the computational effort.

[5] The original implementation of this example including the (scaled) dynamics, etc can be found in *https://github.com/heyinUCB/Backward-Reachability-Analysis-and-Control-Synthesis*. We used this implementation as a baseline. Compared to this implementation (in SOSOPT with custom bisection) we make use of the GSOSOPT module, i.e. to benchmark the toolboxes. Hence, in this specific case, results (computation time etc.) might differ.

1. Solve for s_1^* with V, γ_1 fixed as

$$s_1(V^* - \gamma_1) - \nabla_x V^* f - l(x) \in \Sigma[x]$$

2. β-step: Solve for β^* and s_2^* with V, γ^* fixed as

$$(\beta^*, s_2^*) \in \underset{\beta \in \mathbb{R}, s_2 \in \Sigma[x]}{\arg\max} \ \beta$$
$$\text{s.t. } s_2(p - \beta) + \gamma_1 - V^* \in \Sigma[x]$$

3. V-step: Solve for V with $s_1^*, s_2^*.\gamma_1, \beta^*$ fixed s.t.

$$\begin{aligned} V - l(x) &\in \Sigma[x] \\ s_1^*(V - \gamma_1) - \nabla_x V f - l(x) &\in \Sigma[x] \\ s_2^*(p - \beta^*) + \gamma_1 - V &\in \Sigma[x] \end{aligned}$$

4. Repeat steps 1–3 for 20 iterations.[6]

The polynomial degrees used in this example can be found in Table 4.

Table 4: Benchmark III polynomials and their degrees.

Polynomial	Type	min deg.	max deg
V	$\mathbb{R}[x]$	2	2
s_1	$\Sigma[x]$ or $\mathbb{R}[x]$	1 or 2	2 or 4
s_2	$\Sigma[x]$ or $\mathbb{R}[x]$	0	1 or 2

[6] We set 20 as a maximum to limit the computational effort.

4 Information About Other Toolboxes

This subsection provides short documentation about the used toolboxes and changes we made to, e.g. measure the wall time spent in the low-level solver. We first provide a list of used versions of the toolboxes. Afterwards, if applicable, for each toolbox, we give a list of changes we made and why. No major changes were incorporated, i.e., we did only modify small things such as wall time measurement!

4.1 Toolbox Modifications

SOSOPT/GSOSOPT

Change	Reason
Within bisection we store ALL iteration info structs.	Otherwise, only the last iterate is stored including computation times.
Add tic-toc time measure in sedumi2mosek.m and extended info struct	To measure time spent in low-level solver only.
Augmented mosek output with info struct.	To get additional information such as feasibility ratio.
Adapted/augmented feasibility check (feascheck.m) with feasibility ratio of mosek solver. Just to make sure the solution is feasible.	

Table 5: Changes in SOSOPT Toolbox.

SOSTOOLS

Change	Reason
Commented out several parts with disp()	Could not be turned off by parameter. Otherwise, time measures would be different.
Add tic-toc time measure around mosekopt and extended info struct	To measure time spent in low-level solver only.

Table 6: Changes in SOSTOOLS Toolbox.

SPOTless

Change	Reason
Augmented Mosek STATUS switch case with "otherwise".	In biscetions an error occurred because the case was not defined (probably newer Mosek versions have more outputs). In that case, SPOTless always canceled the optimization with an error.

Table 7: Changes in SPOTless toolbox.

5 Benchmark Execution

In this subsection we describe a little bit more in detail how the benchmarks were conducted. This includes how we measure the different times and how we compare the test results.

Time Measurements

To compare the efficiency in terms of computation time we define several time measures as defined next. Afterward, we describe how they are measured in different toolboxes.

Measure	Description
Parse time	This is the time spent by a specific toolbox to setup the SOS constraints, and do the transcription from polynomial optimization problem to the SDP and back. This process also includes potential simplification or scaling techniques of certain toolboxes.
Solver time	This is only the time spent in the low-level solver.
Total time	Basically the sum of parse time and solver time.

Table 8: Definition of computation times.

For all toolboxes except CaΣoS, we measure as shown in the following listing.

Listing 1: MATLAB script for polynomial optimization and time measurement

```
% Start measurement using tic
tic;

%Setup constraints
Constraints = [p1 >= 0, p2 >= 0];

%Call toolbox to solve problem
Solution = solve_sos_toolbox(Constraints,options);

%End of measurement with toc
elapsedTime = toc;  % End timing

solverTime = Solution.solvertime;

% get parse time
parseTime = elapsedTime - solverTime;
```

In words: we first measure the total time and afterward, we subtract the solver time to get the parse time. This is done for every subproblem involved. In case of a bisection and/or coordinate descent, we measure the times for each iteration and store them in an array. In the end, we sum up the parse and solver times for each solver to obtain the overall parse time and solver time. Additionally, to the time measure, we potentially store e.g. the sublevel set and Lyapunov/storage function which are the results of the optimisation to compare the results.

For CaΣoS the situation is different. Instead, of parsing each solver in each iteration from scratch the parameterized solver is setup once before the actual optimization procedure. Within, for example, a coordinate descent with bisection, potential parameter are passed to the aforementioned parameterized solver which internally maps the parameter to the underlying solver. Once the underlying solver solved the problem it maps the solution back. Thus, for CaΣoS we have to measure the total parse time different. This is depicted in the following listing for one optimization.

Listing 2: MATLAB script for polynomial optimization and time measurement in CaΣoS

```
Start measurement using tic
tic;

%Setup constraints
Constraints = [p1 >= 0, p2 >= 0];

% setup casos solver
solver = setup_solver(Constraints,options);
elapsedTime_parse = toc;  % End timing

%start the actual optimization
tic
solution = solver(parameter)
elapsedTime_solve = toc;  % End timing

solverTime = Solution.solvertime;

% get parse time
parseTime2 = elapsedTime_solve - solverTime;

% get total parse time
parseTime = elapsedTime_parse + parseTime2;
```

We first measure the time to set up the solver. Within the optimization, we measure the total time spent within the CaΣoS solver instance. Afterward, we subtract the low-solver time from the CaΣoS solver time to obtain the so to say second parse time/call time. Finally, we sum up the parse times to obtain the overall parse time. This is equivalent to the other toolboxes i.e. we measure the total time and subtract the solver time to get the parse time.

In case of a coordinate descent with bisection we measure the times for each iteration and store them in an array. In the end, we sum up the parse and solver times for each solver to obtain the overall parse time and solver time.

References

[1] T. Cunis and J. Olucak, "CaΣos: A nonlinear sum-of-squares optimization suite," 2024. [Online]. Available: https://arxiv.org/abs/2409.18549

[2] A. Chakraborty, P. Seiler, and G. J. Balas, "Nonlinear region of attraction analysis for flight control verification and validation," *Control Engineering Practice*, vol. 19, no. 4, pp. 335–345, 2011.

[3] H. Yin, P. Seiler, and M. Arcak, "Backward Reachability Using Integral Quadratic Constraints for Uncertain Nonlinear Systems," *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 707–712, 2021.

[4] W. Tan, "Nonlinear Control Analysis and Synthesis using Sum-of-Squares Programming," Ph.D. dissertation, University of California, Berkeley, Berkeley, US-CA, 2006.